Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет программной инженерии и компьютерной техники

Дисциплина: Моделирование

УЧЕБНО-ИССЛЕДОВАТЕЛЬСКАЯ РАБОТА №2

по теме: «Марковские модели систем массового обслуживания»

Вариант: 33/73/11

Выполнил:
Михайлов Петр Сергеевич
Группа: P3311

Преподаватель:
Тропченко Андрей Александрович

Санкт-Петербург, 2026

Содержание

1 Цель и задачи работы **3**

2 Постановка задачи и исходные данные **3**

3 Описание систем и способ расчёта **4**
3.1 Фазовое представление распределения Эрланга 4
3.2 Вычислительный метод и воспроизводимость 4

4 Марковские модели исследуемых систем **5**
4.1 Кодирование и перечень состояний 5
4.2 Размеченные графы переходов 5
4.3 Матрицы интенсивностей переходов 6

5 Стационарные вероятности состояний **7**
5.1 Независимая аналитическая проверка распределений 9

6 Расчёт характеристик через вероятности состояний **9**
6.1 Нагрузка, загрузка, очереди, потери и производительность 9
6.2 Времена ожидания и пребывания: прямой расчёт 10
6.3 Явные формулы времени для данного варианта 11
6.4 Значения характеристик (форма 2) 12
6.5 Проверка результатов 13

7 Расчёт в WinMark и подтверждение результатов **14**
7.1 Модели, формулы и результаты в исходной точке 14
7.2 Сверка с независимым расчётом 14
7.3 Снимки окон WinMark 15
7.4 Параметры варьирования и экспорт графиков 16

8 Сравнительный анализ систем **18**
8.1 Варьирование интенсивности входного потока 18
8.2 Объяснение различий и влияние накопителей 21
8.3 Влияние выбора прибора с обслуживанием E_2 22

9 Выбор наилучшего варианта организации системы **23**

10 Выводы **23**

Список использованных материалов **24**

A Полная численная матрица второй системы **25**

B Программа расчёта всех характеристик **26**

C Протокол фактического выполнения **30**

D Протокол сверки HTML-отчётов WinMark **31**

1. Цель и задачи работы

Цель работы --- изучить метод марковских случайных процессов с непрерывным временем и применить его для исследования систем массового обслуживания (СМО) с однородным потоком заявок.

Для достижения цели необходимо описать две заданные системы, закодировать их состояния, построить размеченные графы переходов и матрицы интенсивностей, найти стационарные вероятности и рассчитать через них характеристики обслуживания. Затем требуется проверить результаты фундаментальными зависимостями, исследовать влияние интенсивности входного потока и выбрать лучшую организацию системы по критерию варианта.

2. Постановка задачи и исходные данные

Номер варианта: $N_1/N_2/N_G = 33/73/11$. По таблице 1 задания [1] выбираются структуры систем, по таблице 2 --- параметры нагрузки. Поскольку $33 + 73 = 106$ --- чётное число, критерий определяется строкой $N_2 = 73$: **(в) максимальная загрузка системы**.

Таблица 1. Исходные данные варианта 33/73/11

Параметр	СИСТЕМА_1	СИСТЕМА_2
Вариант структуры	33	73
Число приборов c	2	3
Ёмкости накопителей	$2/1$	$1/0/1$
Максимум заявок у приборов K_i	$(3, 2)$	$(2, 1, 2)$
Закон обслуживания	экспоненциальный в обоих приборах	Эрланга E_2 в П1; экспоненциальный в П2 и П3
Вероятности направления q_i	$(0{,}75; 0{,}25)$	$(0{,}75; 0{,}20; 0{,}05)$
Интенсивности λ_i, с^{-1}	$(0{,}45; 0{,}15)$	$(0{,}45; 0{,}12; 0{,}03)$
Общий входной поток λ	$0{,}6\,\text{с}^{-1}$	
Среднее обслуживание b	5 с в каждом приборе обеих систем	
Интенсивность экспоненциального обслуживания $\mu = 1/b$	$0{,}2\,\text{с}^{-1}$	
Критерий эффективности	максимум суммарной загрузки R	

У двухприборной системы вероятность направления ко второму прибору равна сумме двух последних вероятностей таблицы 2: $q_2 = 0{,}20 + 0{,}05 = 0{,}25$. Ёмкость накопителя *не включает* обслуживаемую заявку, поэтому $K_i = r_i + 1$, где r_i --- число мест ожидания.

Обозначение $3(E_2)$ по заданию относится к *одному* из трёх приборов. Выбран прибор П1, принимающий наибольшую долю входного потока. В остальных приборах сохраняется экспоненциальное обслуживание. В основных расчётах и при варьировании λ этот выбор фиксирован. Влияние выбора прибора отдельно проверено в разделе 8.3.

3. Описание систем и способ расчёта

Входной поток --- простейший. Каждая заявка независимо направляется к прибору Пi с вероятностью q_i, поэтому отдельные входные потоки являются независимыми пуассоновскими потоками с $\lambda_i = q_i\lambda$. Если выбранный прибор свободен, обслуживание начинается сразу; иначе заявка занимает свободное место его накопителя. При заполненном накопителе она теряется. Перенаправления к другим приборам нет.

Дисциплина очереди --- FIFO; обслуживание не прерывается. Длительности обслуживания разных заявок и входные потоки независимы. Потерянные заявки не возвращаются. Мгновенная передача очередной заявки из накопителя в прибор не требует отдельного состояния.

3.1. Фазовое представление распределения Эрланга

В П1 второй системы время обслуживания является суммой двух независимых экспоненциальных фаз:

$$B = B^{(1)} + B^{(2)}, \qquad B^{(f)} \sim \mathrm{Exp}(\nu), \qquad \nu = \frac{2}{b} = 0{,}4\,\text{с}^{-1}.$$

Следовательно, $\mathsf{E}B = 2/\nu = 5$ с, $\mathrm{Var}\,B = 2/\nu^2 = 12{,}5$ с^2, коэффициент вариации равен $1/\sqrt{2}$. Две фазы выполняются *последовательно одним прибором*: одновременно две заявки в нём не обслуживаются. Переход между фазами не означает уход заявки. Включение номера текущей фазы в состояние делает процесс марковским [2].

3.2. Вычислительный метод и воспроизводимость

Обе полные модели рассчитаны в программе MARK (WinMark): введены элементы и матрицы интенсивностей, получены стационарные вероятности, вычислены характеристики по 24 формулам для СИСТЕМА_1 и 32 формулам для СИСТЕМА_2, выполнено варьирование интенсивностей и построены графики. Снимки окон программы, результаты сверки и экспортированные графики приведены в разделе 7; полные HTML-отчёты входят в электронное приложение [4, 5].

Независимая программа на Python 3.12.14 со стандартной библиотекой строит генераторы и решает стационарные уравнения методом Гаусса с выбором главного элемента. Времена вычисляются прямым усреднением условных времён по состояниям. Программа используется для проверки WinMark, контрольных экспериментов и уточнения точки пересечения кривых. Имитационное моделирование и оценивание по случайным выборкам не применяются. Исходный код и фактический протокол приведены в приложениях B и C.

Полные модели содержат 12 и 30 состояний. На странице 7 задания допускается обоснованное изменение структуры, если состояний больше 20. В данном расчёте WinMark обрабатывает все 30 состояний: это подтверждается матрицей и вероятностями в `report_system2.html` и файлом `system2_73.mrk`. Поэтому число приборов и ёмкости накопителей сохранены точно по варианту; сокращённая модель не строилась.

Исходный код, модели WinMark, результаты расчётов и материалы отчёта размещены в репозитории: Axe-On-You/Modeling, папка «Лабораторная 2».

4. Марковские модели исследуемых систем

4.1. Кодирование и перечень состояний

Для СИСТЕМА_1 состояние (n_1, n_2) задаёт число заявок в каждом узле вместе с обслуживаемой:

$$n_1 \in \{0,1,2,3\}, \qquad n_2 \in \{0,1,2\}, \qquad E_{3n_1+n_2} = (n_1, n_2).$$

Всего $4 \cdot 3 = 12$ состояний, от $E_0 = (0,0)$ до $E_{11} = (3,2)$.

Для СИСТЕМА_2 локальное состояние П1 кодируется символом

$$a \in \{0, 1_1, 1_2, 2_1, 2_2\}.$$

Здесь 0 означает свободный прибор, n_f --- n заявок в узле и обслуживание текущей заявки в фазе f. Например, 2_2 означает одну заявку на второй фазе обслуживания и одну в накопителе. Пусть $j(a) = 0,1,2,3,4$ в указанном порядке. Тогда

$$E_{6j(a)+3n_2+n_3} = (a, n_2, n_3), \qquad n_2 \in \{0,1\}, \quad n_3 \in \{0,1,2\}.$$

Всего $5 \cdot 2 \cdot 3 = 30$ состояний. Например, $E_8 = (1_1, 0, 2)$, $E_{29} = (2_2, 1, 2)$. Полный перечень *всех* состояний обеих систем с вероятностями приведён в таблице 2.

4.2. Размеченные графы переходов

В первой системе поступление к Пi увеличивает n_i на единицу с интенсивностью λ_i, если $n_i < K_i$. Завершение обслуживания уменьшает n_i на единицу с интенсивностью μ, если $n_i > 0$. Одновременное изменение двух координат не допускается. Полный граф представлен на рисунке 1.

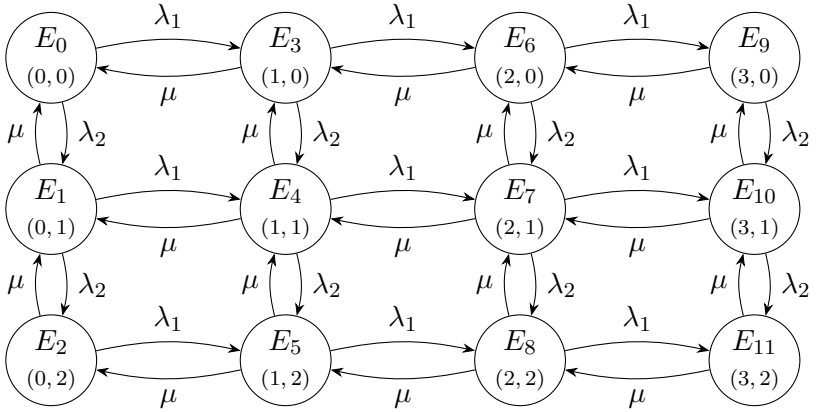


Рис. 1. Размеченный граф СИСТЕМА_1: $\lambda_1 = 0{,}45$, $\lambda_2 = 0{,}15$, $\mu = 0{,}2\,\text{с}^{-1}$.

Для П1 второй системы локальные переходы имеют вид

$$0 \xrightarrow{\lambda_1} 1_1, \quad 1_1 \xrightarrow{\nu} 1_2, \quad 1_2 \xrightarrow{\nu} 0, \quad 1_1 \xrightarrow{\lambda_1} 2_1, \quad 1_2 \xrightarrow{\lambda_1} 2_2, \quad 2_1 \xrightarrow{\nu} 2_2, \quad 2_2 \xrightarrow{\nu} 1_1.$$

Последняя дуга означает завершение обслуживания текущей заявки и начало обслуживания ожидающей в первой фазе. Поступление в состояниях $2_1, 2_2$ теряется. У П2 и П3 действуют обычные экспоненциальные переходы при своих ограничениях ёмкости.

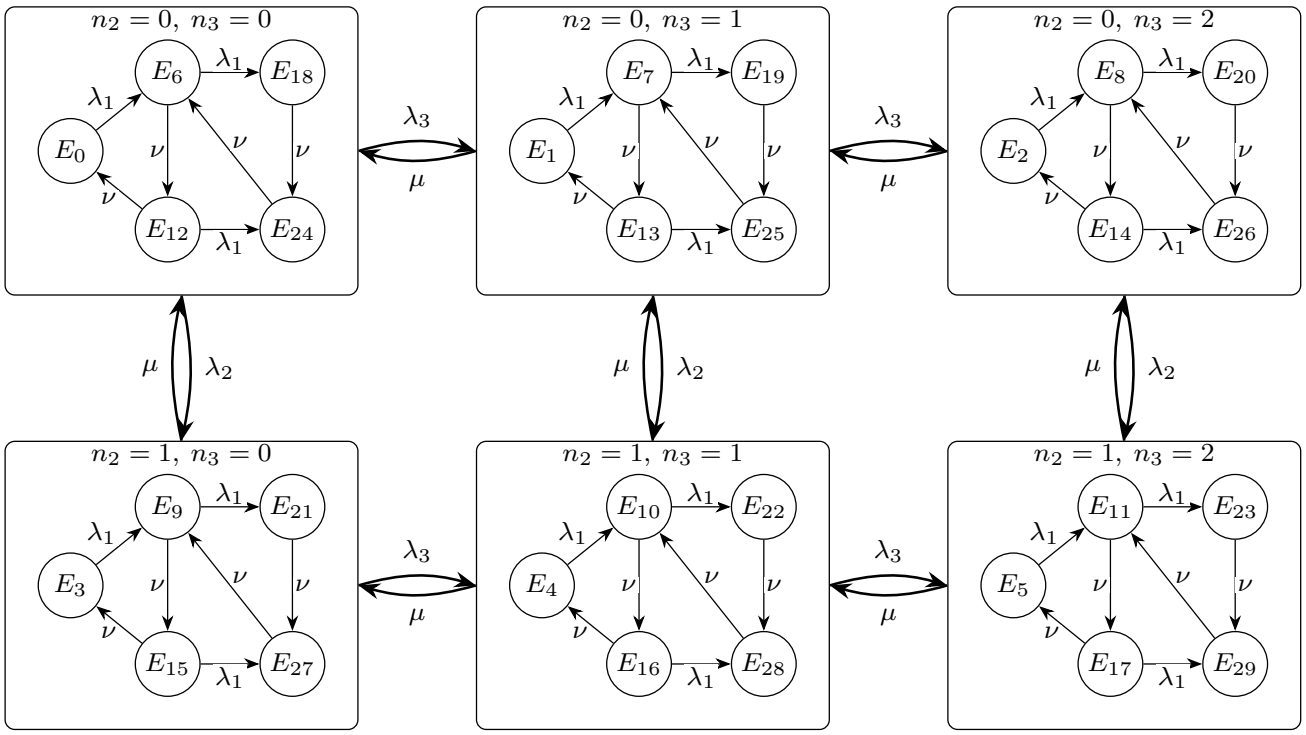


Рис. 2. Полный размеченный граф СИСТЕМА_2 в блочном представлении: показаны все 30 вершин. Внутри блоков $\lambda_1 = 0{,}45$, $\nu = 0{,}4$; между блоками $\lambda_2 = 0{,}12$, $\lambda_3 = 0{,}03$, $\mu = 0{,}2\,\text{с}^{-1}$.

Каждая стрелка между блоками рисунка 2 обозначает пять дуг между *соответствующими* вершинами с одинаковым состоянием a прибора П1. Точная расшифровка:

$$\begin{aligned} E_{6j+3n_2+n_3} &\xrightarrow{\lambda_3} E_{6j+3n_2+n_3+1}, && n_3 < 2, \\ E_{6j+3n_2+n_3} &\xrightarrow{\mu} E_{6j+3n_2+n_3-1}, && n_3 > 0, \\ E_{6j+n_3} &\xrightarrow{\lambda_2} E_{6j+3+n_3}, && n_2 = 0, \\ E_{6j+3+n_3} &\xrightarrow{\mu} E_{6j+n_3}, && n_2 = 1, \end{aligned} \qquad j = 0, \ldots, 4.$$

Таким образом, блочное представление задаёт все дуги исходной модели и не объединяет различные марковские состояния.

Отказы не меняют состояние, поэтому отдельные петли отказов в графах и положительные слагаемые от отказов на диагонали генератора не вводятся. Потери учитываются отдельно через вероятности заполнения соответствующих узлов.

4.3. Матрицы интенсивностей переходов

Используется строковое соглашение: g_{ij} --- интенсивность перехода из E_i в E_j, $i \neq j$, а $g_{ii} = -\sum_{j \neq i} g_{ij}$. Все интенсивности далее измеряются в с^{-1}.

Полная численная матрица первой системы:

$$G_1 = \begin{pmatrix} -0{,}60 & 0{,}15 & 0 & 0{,}45 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0{,}20 & -0{,}80 & 0{,}15 & 0 & 0{,}45 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0{,}20 & -0{,}65 & 0 & 0 & 0{,}45 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0{,}20 & 0 & 0 & -0{,}80 & 0{,}15 & 0 & 0{,}45 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0{,}20 & 0 & 0{,}20 & -1{,}00 & 0{,}15 & 0 & 0{,}45 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0{,}20 & 0 & 0{,}20 & -0{,}85 & 0 & 0 & 0{,}45 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0{,}20 & 0 & 0 & -0{,}80 & 0{,}15 & 0 & 0{,}45 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0{,}20 & 0 & 0{,}20 & -1{,}00 & 0{,}15 & 0 & 0{,}45 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0{,}20 & 0 & 0{,}20 & -0{,}85 & 0 & 0 & 0{,}45 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0{,}20 & 0 & 0 & -0{,}35 & 0{,}15 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0{,}20 & 0 & 0{,}20 & -0{,}55 & 0{,}15 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0{,}20 & 0 & 0{,}20 & -0{,}40 \end{pmatrix}. \tag{1}$$

Для второй системы введём локальные генераторы в порядке состояний, принятом выше:

$$A = \begin{pmatrix} -\lambda_1 & \lambda_1 & 0 & 0 & 0 \\ 0 & -(\lambda_1+\nu) & \nu & \lambda_1 & 0 \\ \nu & 0 & -(\lambda_1+\nu) & 0 & \lambda_1 \\ 0 & 0 & 0 & -\nu & \nu \\ 0 & \nu & 0 & 0 & -\nu \end{pmatrix}, \quad B = \begin{pmatrix} -\lambda_2 & \lambda_2 \\ \mu & -\mu \end{pmatrix},$$

$$C = \begin{pmatrix} -\lambda_3 & \lambda_3 & 0 \\ \mu & -(\lambda_3+\mu) & \lambda_3 \\ 0 & \mu & -\mu \end{pmatrix}, \qquad (\lambda_1,\lambda_2,\lambda_3,\mu,\nu) = (0{,}45; 0{,}12; 0{,}03; 0{,}2; 0{,}4).$$

Полная матрица размером 30×30 задаётся кронекеровой суммой

$$G_2 = A \otimes I_6 + I_5 \otimes B \otimes I_3 + I_{10} \otimes C, \tag{2}$$

где I_k --- единичная матрица, $\otimes$ --- кронекерово произведение. Для явной блочной записи положим $D = B \otimes I_3 + I_2 \otimes C$, $I = I_6$. Тогда

$$G_2 = \begin{pmatrix} D - 0{,}45I & 0{,}45I & 0 & 0 & 0 \\ 0 & D - 0{,}85I & 0{,}4I & 0{,}45I & 0 \\ 0{,}4I & 0 & D - 0{,}85I & 0 & 0{,}45I \\ 0 & 0 & 0 & D - 0{,}4I & 0{,}4I \\ 0 & 0{,}4I & 0 & 0 & D - 0{,}4I \end{pmatrix}.$$

Все 30 строк этой же матрицы в численном разреженном представлении приведены в приложении А. Например, из $E_8 = (1_1, 0, 2)$ возможны переходы $E_8 \to E_{20}$ с $0{,}45$, $E_8 \to E_{11}$ с $0{,}12$, $E_8 \to E_{14}$ с $0{,}4$, $E_8 \to E_7$ с $0{,}2$, поэтому $g_{88} = -1{,}17$.

5. Стационарные вероятности состояний

Обе цепи конечны и неприводимы: из любого состояния можно завершить обслуживание всех заявок, а из пустого состояния достижимы все допустимые состояния. При положительных

интенсивностях это обеспечивает единственное стационарное распределение независимо от того, превышает ли предлагаемая нагрузка единицу [2, 3].

Искомый строковый вектор p определяется системой

$$pG = 0, \qquad \sum_{s} p_s = 1. \tag{3}$$

В независимом расчёте на Python матрица транспонируется, одно из уравнений баланса заменяется нормировкой, после чего решается невырожденная СЛАУ. В таблице приведены уточнённые вероятности, согласующиеся с шестизначной печатью WinMark. Значения таблицы 2 получены *из полной матрицы каждой системы* при $\lambda = 0{,}6\,\text{с}^{-1}$.

Таблица 2. Стационарные вероятности состояний (форма 1)

	СИСТЕМА_1		СИСТЕМА_2	
Номер	Обозначение	Вероятность	Обозначение	Вероятность
0	$E_0 = (0,0)$	0,02194740	$E_0 = (0,0,0)$	0,04776358
1	$E_1 = (0,1)$	0,01646055	$E_1 = (0,0,1)$	0,00716454
2	$E_2 = (0,2)$	0,01234541	$E_2 = (0,0,2)$	0,00107468
3	$E_3 = (1,0)$	0,04938166	$E_3 = (0,1,0)$	0,02865815
4	$E_4 = (1,1)$	0,03703624	$E_4 = (0,1,1)$	0,00429872
5	$E_5 = (1,2)$	0,02777718	$E_5 = (0,1,2)$	0,00064481
6	$E_6 = (2,0)$	0,11110873	$E_6 = (1_1,0,0)$	0,11418481
7	$E_7 = (2,1)$	0,08333155	$E_7 = (1_1,0,1)$	0,01712772
8	$E_8 = (2,2)$	0,06249866	$E_8 = (1_1,0,2)$	0,00256916
9	$E_9 = (3,0)$	0,24999464	$E_9 = (1_1,1,0)$	0,06851089
10	$E_{10} = (3,1)$	0,18749598	$E_{10} = (1_1,1,1)$	0,01027663
11	$E_{11} = (3,2)$	0,14062199	$E_{11} = (1_1,1,2)$	0,00154149
12	---	---	$E_{12} = (1_2,0,0)$	0,05373403
13	---	---	$E_{13} = (1_2,0,1)$	0,00806010
14	---	---	$E_{14} = (1_2,0,2)$	0,00120902
15	---	---	$E_{15} = (1_2,1,0)$	0,03224042
16	---	---	$E_{16} = (1_2,1,1)$	0,00483606
17	---	---	$E_{17} = (1_2,1,2)$	0,00072541
18	---	---	$E_{18} = (2_1,0,0)$	0,12845792
19	---	---	$E_{19} = (2_1,0,1)$	0,01926869
20	---	---	$E_{20} = (2_1,0,2)$	0,00289030
21	---	---	$E_{21} = (2_1,1,0)$	0,07707475
22	---	---	$E_{22} = (2_1,1,1)$	0,01156121
23	---	---	$E_{23} = (2_1,1,2)$	0,00173418
24	---	---	$E_{24} = (2_2,0,0)$	0,18890870
25	---	---	$E_{25} = (2_2,0,1)$	0,02833630
26	---	---	$E_{26} = (2_2,0,2)$	0,00425045
27	---	---	$E_{27} = (2_2,1,0)$	0,11334522
28	---	---	$E_{28} = (2_2,1,1)$	0,01700178
29	---	---	$E_{29} = (2_2,1,2)$	0,00255027
$\sum$		1,00000000		1,00000000

Вероятности округлены независимо; нормировка проверяется до округления.

5.1. Независимая аналитическая проверка распределений

Благодаря независимому разделению пуассоновского потока узлы работают независимо. Поэтому полные вероятности должны совпадать с произведениями локальных вероятностей. Это используется как *проверка*, а не замена решения (3).

Для экспоненциального узла с максимумом K заявок и $y = \lambda_i b$:

$$p_n^{\text{loc}} = \frac{y^n}{\sum_{h=0}^{K} y^h}, \qquad n = 0, \dots, K.$$

Для узла $M/E_2/1$ с одним местом ожидания, $t = \lambda_1/\nu$, локальные вероятности в порядке $(0, 1_1, 1_2, 2_1, 2_2)$ равны

$$\frac{\big(1,\ t(1+t),\ t,\ t^2(1+t),\ t^2(2+t)\big)}{1 + 2t + 4t^2 + 2t^3}.$$

Это выражение следует из пяти локальных уравнений баланса, например $\lambda_1 p_0 = \nu p_{1_2}$ и $(\lambda_1 + \nu)p_{1_2} = \nu p_{1_1}$. Для заданной нагрузки $t = 1{,}125$.

Таблица 3. Локальные стационарные вероятности в порядке локальных состояний

Система	Прибор	p_0	p_1	p_2	p_3	p_4
1	П1	0,050753	0,114195	0,256939	0,578113	---
1	П2	0,432432	0,324324	0,243243	---	---
2	П1	0,089604	0,214211	0,100805	0,240987	0,354393
2	П2	0,625000	0,375000	---	---	---
2	П3	0,852878	0,127932	0,019190	---	---

У Эрланга индексы $0, 1, 2, 3, 4$ в этой таблице соответствуют $(0, 1_1, 1_2, 2_1, 2_2)$; у экспоненциальных приборов индекс равен числу заявок. Максимальные расхождения произведений локальных вероятностей с полными распределениями: $8{,}67 \cdot 10^{-17}$ для СИСТЕМА_1 и $3{,}47 \cdot 10^{-17}$ для СИСТЕМА_2.

6. Расчёт характеристик через вероятности состояний

6.1. Нагрузка, загрузка, очереди, потери и производительность

Пусть s --- полное состояние, $n_i(s)$ --- число заявок у Пi, p_s --- его стационарная вероятность, $x^+ = \max(x, 0)$. Тогда

$$y_i = \lambda_i b, \qquad \rho_i = \sum_s p_s \mathbf{1}_{\{n_i(s)>0\}}, \tag{4}$$

$$l_i = \sum_s p_s \big(n_i(s) - 1\big)^+, \qquad m_i = \sum_s p_s n_i(s), \tag{5}$$

$$\pi_i = \sum_s p_s \mathbf{1}_{\{n_i(s)=K_i\}}, \qquad X_i = \sum_s p_s d_i(s). \tag{6}$$

Здесь $d_i(s)$ --- интенсивность *завершения обслуживания заявки* в состоянии s: для экспоненциального прибора $d_i(s) = \mu$ при $n_i > 0$, иначе ноль; для П1 второй системы $d_1(s) = \nu$ только при фазе 2, иначе ноль. В частности,

$$X_1^{(2)} = \nu\, P\{a \in \{1_2, 2_2\}\},$$

а завершение первой фазы в производительность не входит.

Суммарные показатели определяются как

$$Y = \sum_i y_i = \lambda b = 3, \quad R = \sum_i \rho_i, \quad L = \sum_i l_i, \quad M = \sum_i m_i, \quad X = \sum_i X_i, \tag{7}$$

$$\pi = \sum_i q_i \pi_i, \qquad \bar{\rho} = R/c. \tag{8}$$

R --- среднее число одновременно работающих приборов, $\bar{\rho}$ --- средняя доля занятого времени одного прибора. В форме 2 задания приводится суммарная загрузка, поэтому критерием сравнения принят максимум R. Дополнительно рассчитывается $\bar{\rho}$, чтобы учитывать различное число приборов. Вероятности потерь *взвешиваются* по долям входного потока, а не складываются без весов.

6.2. Времена ожидания и пребывания: прямой расчёт

Время считается для **принятых** заявок, включая заявки с нулевым ожиданием. Обозначим $A_i = \{s : n_i(s) < K_i\}$ и $a_i = \sum_{s \in A_i} p_s = 1 - \pi_i$.

Пуассоновская поступающая заявка видит состояние s с вероятностью p_s; условно на принятие --- с вероятностью p_s / a_i для $s \in A_i$. Это свойство поступлений PASTA (Poisson Arrivals See Time Averages) позволяет усреднять время по состояниям в момент поступления.

При k_i последовательных экспоненциальных фазах ($k_i = 1$ у экспоненциального прибора, $k_i = 2$ у Эрланга) среднее время до завершения текущего обслуживания в фазе f равно

$$r_i(s) = \frac{k_i - f + 1}{k_i / b}.$$

Если новая заявка видит $n_i \geq 1$ заявок в узле, перед ней находятся остаток текущего обслуживания и $n_i - 1$ полных обслуживаний. Следовательно,

$$\tau_i(s) = \begin{cases} 0, & n_i = 0, \\ r_i(s) + (n_i - 1)b, & 1 \leq n_i < K_i, \end{cases} \qquad \upsilon_i(s) = \tau_i(s) + b. \tag{9}$$

Здесь τ_i --- условное среднее ожидание, υ_i --- условное среднее пребывание. Прямое усреднение даёт

$$\boxed{w_i = \frac{\sum_{s \in A_i} p_s \tau_i(s)}{a_i}, \qquad u_i = \frac{\sum_{s \in A_i} p_s \upsilon_i(s)}{a_i}.} \tag{10}$$

Формулы (9)--(10) являются основным способом расчёта времени. Они не используют отношений l_i / X_i и m_i / X_i. Деление на a_i обусловлено условием принятия заявки; потерянные заявки не включаются в знаменатель.

Для всей системы вероятность направления к Пi среди принятых заявок равна

$$\omega_i = \frac{q_i a_i}{\sum_j q_j a_j}, \qquad W = \sum_i \omega_i w_i, \qquad U = \sum_i \omega_i u_i.$$

Применять исходные q_i как веса времён нельзя, поскольку доли потерянных заявок у приборов различаются.

6.3. Явные формулы времени для данного варианта

Для первой системы введём маргинальные вероятности

$$\alpha_n = \sum_{m=0}^{2} p^{(1)}_{3n+m}, \qquad \beta_m = \sum_{n=0}^{3} p^{(1)}_{3n+m}.$$

Экспоненциальное обслуживание не имеет последействия, поэтому остаток любого текущего обслуживания равен в среднем b. Состояния с заполненным узлом исключаются. Из (10) получаем

$$w_1^{(1)} = \frac{b\alpha_1 + 2b\alpha_2}{\alpha_0 + \alpha_1 + \alpha_2} = 7{,}443609 \text{ с}, \tag{11}$$

$$u_1^{(1)} = \frac{b\alpha_0 + 2b\alpha_1 + 3b\alpha_2}{\alpha_0 + \alpha_1 + \alpha_2} = 12{,}443609 \text{ с}, \tag{12}$$

$$w_2^{(1)} = \frac{b\beta_1}{\beta_0 + \beta_1} = 2{,}142857 \text{ с}, \tag{13}$$

$$u_2^{(1)} = \frac{b\beta_0 + 2b\beta_1}{\beta_0 + \beta_1} = 7{,}142857 \text{ с}. \tag{14}$$

Для второй системы обозначим

$$\gamma_a = \sum_{n_2=0}^{1}\sum_{n_3=0}^{2} p^{(2)}_{6a+3n_2+n_3}, \qquad a = 0, \ldots, 4,$$

$$\delta_{n_2} = \sum_{a=0}^{4}\sum_{n_3=0}^{2} p^{(2)}_{6a+3n_2+n_3}, \qquad \varepsilon_{n_3} = \sum_{a=0}^{4}\sum_{n_2=0}^{1} p^{(2)}_{6a+3n_2+n_3}.$$

При поступлении к П1 заявка принимается только в состояниях $0, 1_1, 1_2$. В 1_1 ей нужно ждать две оставшиеся фазы, то есть 5 с; в 1_2 --- одну фазу, то есть $2{,}5$ с. Поэтому

$$w_1^{(2)} = \frac{b\gamma_1 + (b/2)\gamma_2}{\gamma_0 + \gamma_1 + \gamma_2} = 3{,}269896 \text{ с}, \tag{15}$$

$$u_1^{(2)} = \frac{b\gamma_0 + 2b\gamma_1 + (3b/2)\gamma_2}{\gamma_0 + \gamma_1 + \gamma_2} = 8{,}269896 \text{ с}. \tag{16}$$

П2 не имеет накопителя: принятая заявка всегда поступает к свободному прибору, поэтому $w_2^{(2)} = 0$, $u_2^{(2)} = b = 5$ с. Для П3:

$$w_3^{(2)} = \frac{b\varepsilon_1}{\varepsilon_0 + \varepsilon_1} = 0{,}652174 \text{ с}, \qquad u_3^{(2)} = \frac{b\varepsilon_0 + 2b\varepsilon_1}{\varepsilon_0 + \varepsilon_1} = 5{,}652174 \text{ с}.$$

При эрланговском обслуживании остаточное время текущего обслуживания зависит от номера фазы и в общем случае не равно b. Это учтено в формулах, в отличие от замены эрланговского обслуживания одним экспоненциальным.

6.4. Значения характеристик (форма 2)

В таблице 4 приведены результаты независимого расчёта, сверенные с WinMark, при исходной интенсивности $\lambda = 0{,}6\,\text{с}^{-1}$. Во всех строках «Сумм.» временные характеристики усредняются по принятому потоку, а вероятности потерь --- по входному потоку. Для X_i используется интенсивность завершений $d_i(s)$, определённая выше.

Таблица 4. Характеристики систем при исходной нагрузке (форма 2)

Характеристика	Прибор	Расчётная формула	СИСТ.1	СИСТ.2
Нагрузка	П1	$y_i = \lambda_i b$	2,250000	2,250000
	П2		0,750000	0,600000
	П3		---	0,150000
	Сумм.	$Y = \sum_i y_i$	3,000000	3,000000
Загрузка	П1	$\rho_i = \sum_s p_s \mathbf{1}_{n_i>0}$	0,949247	0,910396
	П2		0,567568	0,375000
	П3		---	0,147122
	Сумм.	$R = \sum_i \rho_i$	1,516814	1,432517
Длина очереди	П1	$l_i = \sum_s p_s (n_i - 1)^+$	1,413164	0,595380
	П2		0,243243	0,000000
	П3		---	0,019190
	Сумм.	$L = \sum_i l_i$	1,656407	0,614570
Число заявок	П1	$m_i = \sum_s p_s n_i$	2,362411	1,505775
	П2		0,810811	0,375000
	П3		---	0,166311
	Сумм.	$M = \sum_i m_i$	3,173222	2,047087
Время ожидания, с	П1	$w_i = \sum_{s \in A_i} p_s \tau_i(s) / a_i$	7,443609	3,269896
	П2		2,142857	0,000000
	П3		---	0,652174
	Сумм.	$W = \sum_i \omega_i w_i$	5,460153	2,145069
Время пребывания, с	П1	$u_i = \sum_{s \in A_i} p_s \upsilon_i(s) / a_i$	12,443609	8,269896
	П2		7,142857	5,000000
	П3		---	5,652174
	Сумм.	$U = \sum_i \omega_i u_i$	10,460153	7,145069
Вероятность потери	П1	$\pi_i = \sum_s p_s \mathbf{1}_{n_i = K_i}$	0,578113	0,595380
	П2		0,243243	0,375000
	П3		---	0,019190
	Сумм.	$\pi = \sum_i q_i \pi_i$	0,494395	0,522494
Производительность, с^{-1}	П1	$X_i = \sum_s p_s d_i(s)$	0,189849	0,182079
	П2		0,113514	0,075000
	П3		---	0,029424
	Сумм.	$X = \sum_i X_i$	0,303363	0,286503

Средняя загрузка одного прибора: $\bar{\rho}^{(1)} = 0{,}758407$, $\bar{\rho}^{(2)} = 0{,}477506$. Загрузка П1 составляет $94{,}92\%$ в первой системе и $91{,}04\%$ во второй; это наиболее загруженный узел в обеих конфигурациях.

6.5. Проверка результатов

После расчёта всех величин независимо проверяются следующие зависимости:

$$X_i = \lambda_i(1-\pi_i), \quad X_i = \rho_i/b, \quad \boxed{l_i = X_i w_i, \qquad m_i = X_i u_i}, \tag{17}$$

$$u_i = w_i + b, \qquad M = L + R, \qquad X = \lambda(1-\pi), \quad L = XW, \quad M = XU. \tag{18}$$

Выделенные равенства и их системные аналоги --- формулы Литтла для принятого потока. В этой работе они служат *только проверкой* времён, предварительно найденных через условные времена по состояниям. Равенство $u_i = w_i + b$ проверяет разложение времени пребывания на ожидание и обслуживание.

Например, с округлением до шести знаков:

$$\begin{aligned}
\text{СИСТЕМА_1}: \quad & 0{,}6(1-0{,}494395) \approx 0{,}303363,\\
& 0{,}303363 \cdot 5{,}460153 \approx 1{,}656407,\\
& 0{,}303363 \cdot 10{,}460153 \approx 3{,}173222;\\
\text{СИСТЕМА_2}: \quad & 0{,}6(1-0{,}522494) \approx 0{,}286503,\\
& 0{,}286503 \cdot 2{,}145069 \approx 0{,}614570,\\
& 0{,}286503 \cdot 7{,}145069 \approx 2{,}047087.
\end{aligned}$$

Для П1 второй системы также проверяется равенство общей стационарной вероятности первой и второй фаз: обе равны $0{,}455198$; завершения второй фазы дают $X_1^{(2)} = 0{,}4 \cdot 0{,}455198 = 0{,}182079\,\text{с}^{-1}$.

Таблица 5. Невязки независимого расчёта на Python до округления результатов

Проверка	СИСТЕМА_1	СИСТЕМА_2
Максимум $\lvert\sum_j g_{ij}\rvert$	$5{,}55 \cdot 10^{-17}$	$8{,}33 \cdot 10^{-17}$
Максимум $\lvert(pG)_j\rvert$	$4{,}51 \cdot 10^{-17}$	$3{,}01 \cdot 10^{-17}$
Расхождение с произведением локальных вероятностей	$8{,}67 \cdot 10^{-17}$	$3{,}47 \cdot 10^{-17}$
Максимальное отклонение по всем программным проверкам	$1{,}78 \cdot 10^{-15}$	$1{,}78 \cdot 10^{-15}$

Все вероятности положительны. Максимальное отклонение по проверкам включает нормировку, баланс генератора, стационарные уравнения, независимое произведение распределений, баланс принятия/завершения и формулы Литтла для каждого узла и системы. Проверки повторяются в каждой точке варьирования.

7. Расчёт в WinMark и подтверждение результатов

7.1. Модели, формулы и результаты в исходной точке

Расчёты выполнены в WinMark 27.10.2026 для варианта 33/73/11. Для каждой системы введена собственная матрица: 12×12 для СИСТЕМА_1 и 30×30 для СИСТЕМА_2. Вторая матрица содержит 112 ненулевых внедиагональных элементов. В П1 второй системы используются последовательные фазы с `nu=0.4`; в П2 и П3 --- экспоненциальное обслуживание с `mu=0.2`.

После ввода матрицы выполнены команды «Расчёт → Вероятности» и «Расчёт → Формулы». Введены 24 и 32 выражения для восьми характеристик каждого прибора и всей системы. В WinMark длина очереди обозначена `Q`, вероятность потери --- `Loss`, производительность --- `Perf`; они соответствуют L, π, X в математической части.

Таблица 6. Общие характеристики из HTML-экспорта WinMark при $\lambda = 0{,}6\,\text{с}^{-1}$

Показатель	СИСТЕМА_1	СИСТЕМА_2
Y	3,000000	3,000000
R	1,516814	1,432517
L (`Q`)	1,656407	0,614569
M	3,173222	2,047086
W, с	5,460153	2,145069
U, с	10,460154	7,145069
π (`Loss`)	0,494395	0,522494
X, с^{-1} (`Perf`)	0,303363	0,286503

Времена в WinMark рассчитаны напрямую по состояниям и остаточным фазам (раздел 6.2). Например, для `U1` второй системы к выражению ожидания прибавлено `2/nu`, то есть среднее собственное обслуживание. Отношения `Q/Perf` и `M/Perf` для вычисления времён не используются. Все выражения сохранены в HTML-отчётах.

7.2. Сверка с независимым расчётом

Проверены матрицы, 42 вероятности и 56 характеристик в исходной точке, а также все 40 точек: 2240 значений характеристик и 1680 вероятностей. Выражения из HTML проверены на независимо найденных вероятностях: расхождение с прямым расчётом менее $2{,}2 \cdot 10^{-14}$.

Таблица 7. Максимальные абсолютные расхождения WinMark и Python

Проверяемые значения	СИСТЕМА_1	СИСТЕМА_2
Характеристики в исходной точке	$2{,}98 \cdot 10^{-6}$	$9{,}13 \cdot 10^{-7}$
Вероятности в исходной точке	$4{,}53 \cdot 10^{-7}$	$4{,}95 \cdot 10^{-7}$
Характеристики в 40 точках	$7{,}91 \cdot 10^{-6}$	$7{,}06 \cdot 10^{-6}$
Вероятности в 40 точках	$5{,}20 \cdot 10^{-7}$	$5{,}29 \cdot 10^{-7}$

Расхождения не превышают 10^{-5} и соответствуют точности расчёта и печати WinMark. В таблице 6 сохранены значения именно из экспорта; поэтому последние знаки могут отличаться от таблицы 4. Протокол автоматической сверки приведён в приложении D.

7.3. Снимки окон WinMark

На рисунке 3 приведены исходные снимки окон результатов обеих систем. В каждой колонке сверху вниз показаны загрузка, времена и потери с производительностью. Полные списки вероятностей и всех характеристик доступны в HTML-отчётах электронного приложения.

СИСТЕМА_1 **СИСТЕМА_2**

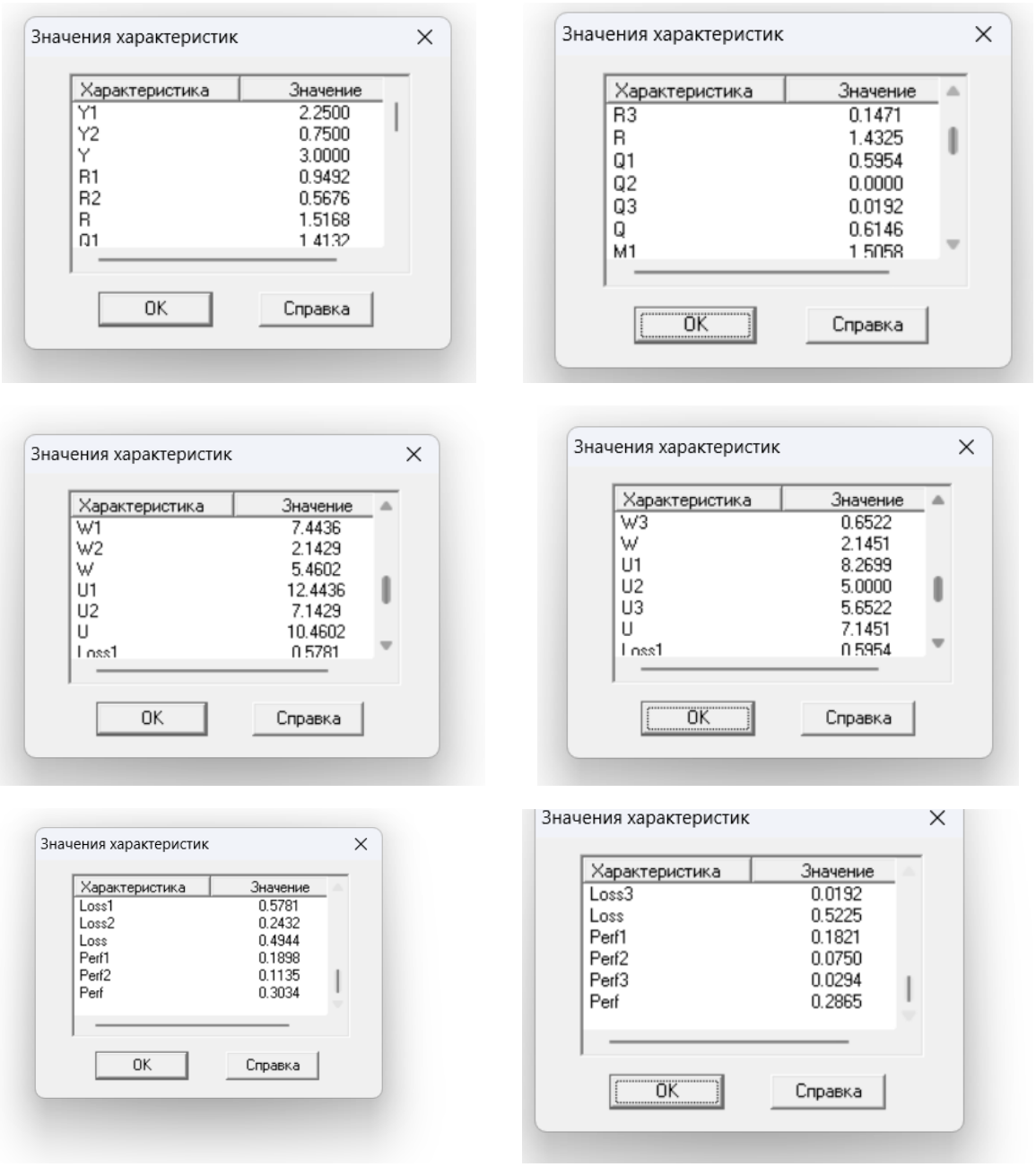


Рис. 3. Результаты непосредственного расчёта в WinMark при исходной нагрузке. Снимки содержат значения R, W, U, `Loss` и `Perf`.

7.4. Параметры варьирования и экспорт графиков

В окне «Данные → Варьируемые элементы» заданы 40 точек. Начальное значение и шаг каждого входного элемента одинаковы; `mu` и `nu` не варьируются. Параметры приведены в таблице 8 и на рисунке 4.

Таблица 8. Варьирование интенсивностей в WinMark

Система	Элемент	Начальное значение	Шаг
1	`l1`	0,0375	0,0375
1	`l2`	0,0125	0,0125
1	`mu`	0,2000	0
2	`l1`	0,0375	0,0375
2	`l2`	0,0100	0,0100
2	`l3`	0,0025	0,0025
2	`mu`	0,2000	0
2	`nu`	0,4000	0

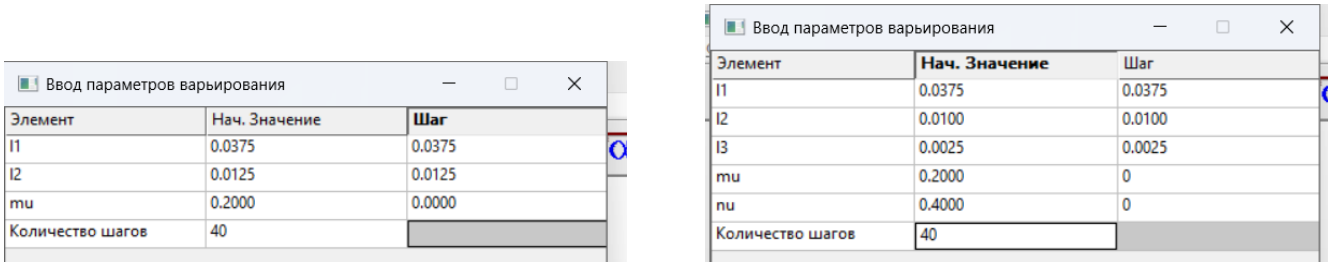


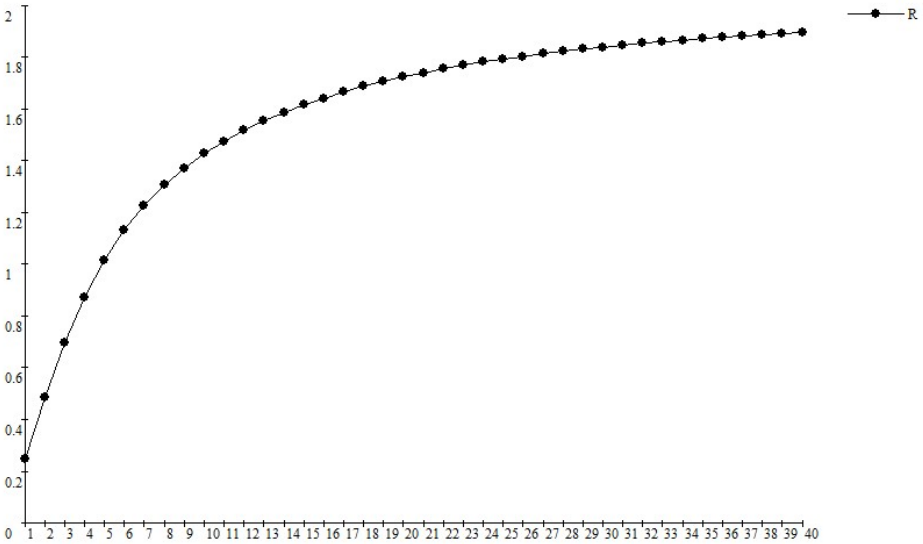
Рис. 4. Окна ввода параметров варьирования: слева СИСТЕМА_1, справа СИСТЕМА_2.

Для шага $h = 1, \ldots, 40$ общая интенсивность равна $\lambda_h = 0{,}05h\,\text{с}^{-1}$. Шаг 12 соответствует $\lambda = 0{,}6\,\text{с}^{-1}$. На графиках, экспортированных самой программой, абсцисса является индексом $j = h - 1$ (от 0 до 39), поэтому $\lambda = 0{,}05(j+1)\,\text{с}^{-1}$. На сравнительных графиках следующего раздела абсциссы переведены в физические значения λ.

Для каждой системы экспортированы шесть отдельных графиков: `R`, `Loss`, `Perf`, `Q`, `W`, `U`. Рисунок 5 показывает исходные графики критерия выбора; все 12 изображений входят в электронное приложение. Перед экспортом характеристик в точке восстановлены исходные элементы и повторно выполнены команды расчёта вероятностей и формул. Поэтому значения в точке в обоих HTML-отчётах относятся к $\lambda = 0{,}6\,\text{с}^{-1}$.

Электронное приложение. В папке `docs` вместе с исходником отчёта находятся `system1/report_system1.html`, `system2/report_system2.html`, папки с графиками и использованные снимки окон. Сохранённые модели обеих систем находятся в `../WinMark/out/system1_33.mrk` и `../WinMark/out/system2_73.mrk`. В HTML ссылки на изображения заменены относительными и явно указана исходная кодировка Windows-1251, чтобы отчёты открывались после переноса папки. Числа, матрицы и формулы оставлены без изменений.

СИСТЕМА_1



СИСТЕМА_2

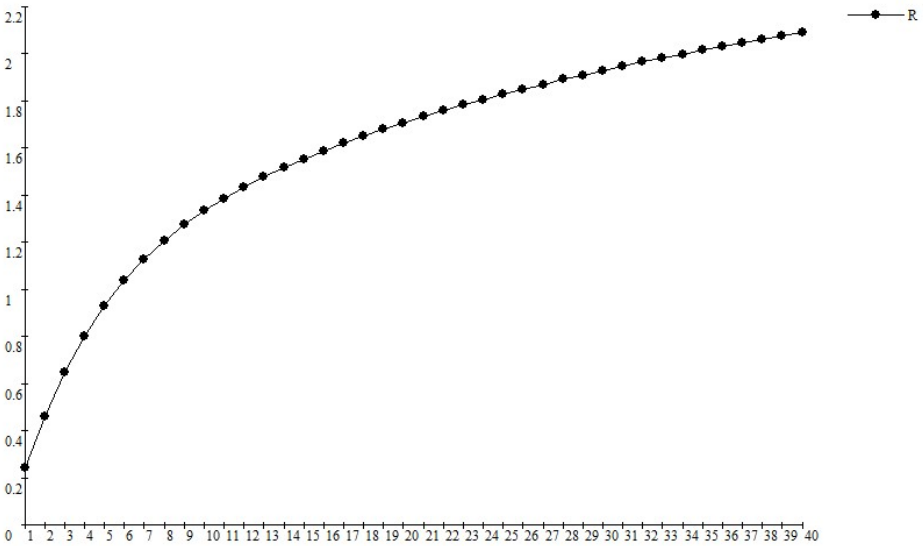


Рис. 5. Графики загрузки R, непосредственно экспортированные из WinMark. По горизонтали показан индекс точки j; соответствующая интенсивность равна $0{,}05(j+1)\,\text{с}^{-1}$.

8. Сравнительный анализ систем

8.1. Варьирование интенсивности входного потока

Интенсивность изменяется от $0{,}05$ до $2{,}00\,\text{с}^{-1}$ с шагом $0{,}05$ (40 точек), а q_i, b, законы обслуживания и накопители сохраняются. В каждой точке WinMark рассчитывает вероятности и все характеристики; результаты независимо проверяются на Python. Кривые ниже построены по числам из HTML-экспорта WinMark. Вертикальная пунктирная линия на графиках показывает исходное значение $\lambda = 0{,}6\,\text{с}^{-1}$.

Таблица 9. Результаты в опорных точках варьирования

	Загрузка R		Потери π		Пребывание U, с	
λ, с^{-1}	СИСТ.1	СИСТ.2	СИСТ.1	СИСТ.2	СИСТ.1	СИСТ.2
0,20	0,8724	0,8030	0,1276	0,1970	8,2205	6,3728
0,40	1,3055	1,2060	0,3473	0,3970	9,7896	6,9134
0,60	1,5168	1,4325	0,4944	0,5225	10,4602	7,1451
1,00	1,7237	1,7068	0,6553	0,6586	11,0349	7,3121
1,50	1,8389	1,9281	0,7548	0,7429	11,3694	7,3784
2,00	1,8954	2,0894	0,8105	0,7911	11,5699	7,4177

При исходной нагрузке первая система имеет большую суммарную загрузку, меньшие потери и большую производительность. При увеличении λ преимущество по R меняется: уточнение на Python между точками варьирования, то есть численное решение уравнения $R^{(1)}(\lambda) = R^{(2)}(\lambda)$, даёт

$$\lambda_* \approx 1{,}083188\,\text{с}^{-1}.$$

На исследованном диапазоне первая система лучше по R при $\lambda < \lambda_*$, вторая --- при $\lambda > \lambda_*$. При одинаковом b в обоих вариантах $R = bX$, поэтому пересечение производительностей происходит в той же точке. Для положительной общей интенсивности пересечение потерь также совпадает с ней. Вывод о победителе относится прежде всего к заданной нагрузке $\lambda = 0{,}6$, а не ко всем возможным нагрузкам.

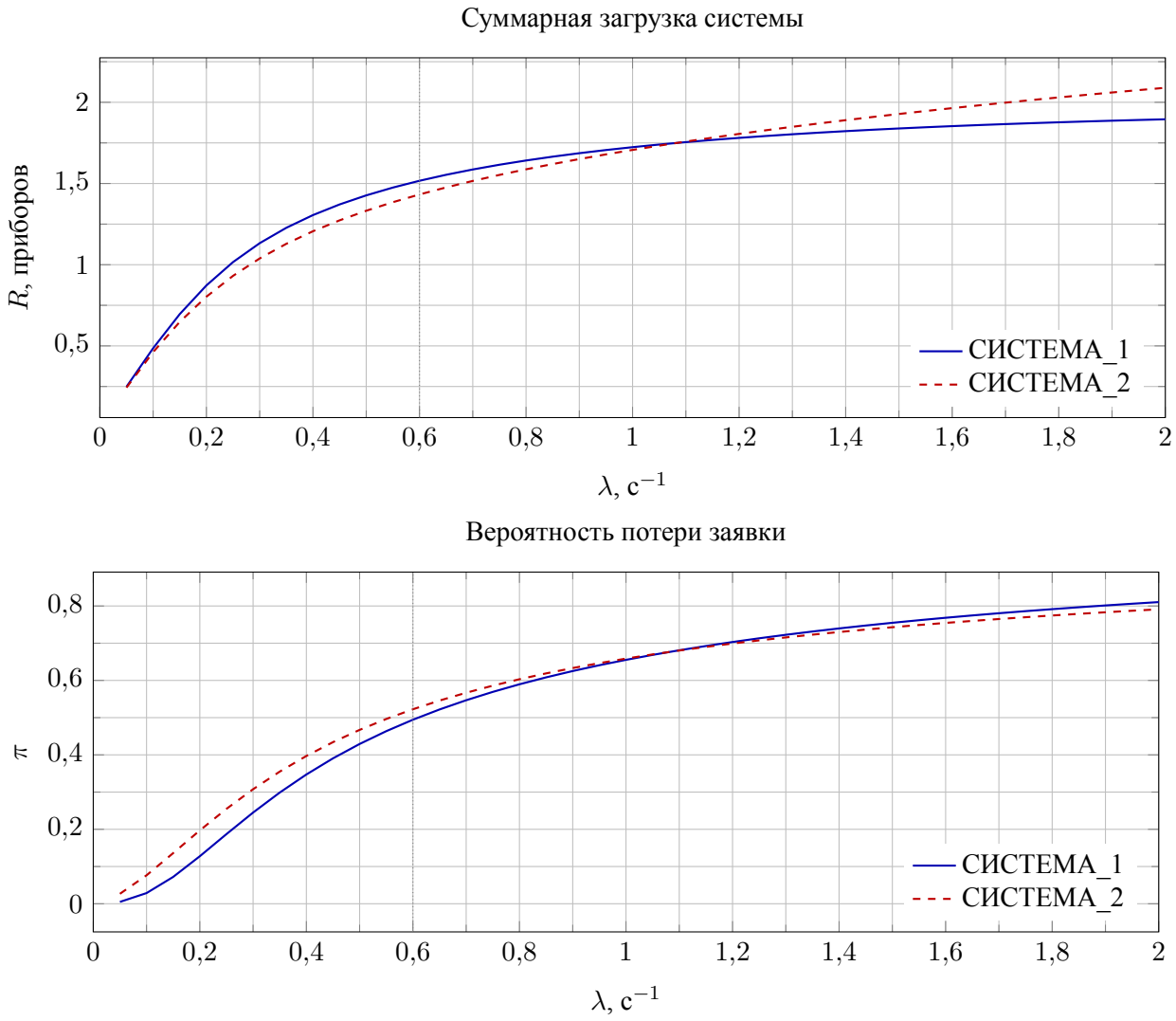


Рис. 6. Влияние входной интенсивности на суммарную загрузку и вероятность потери.

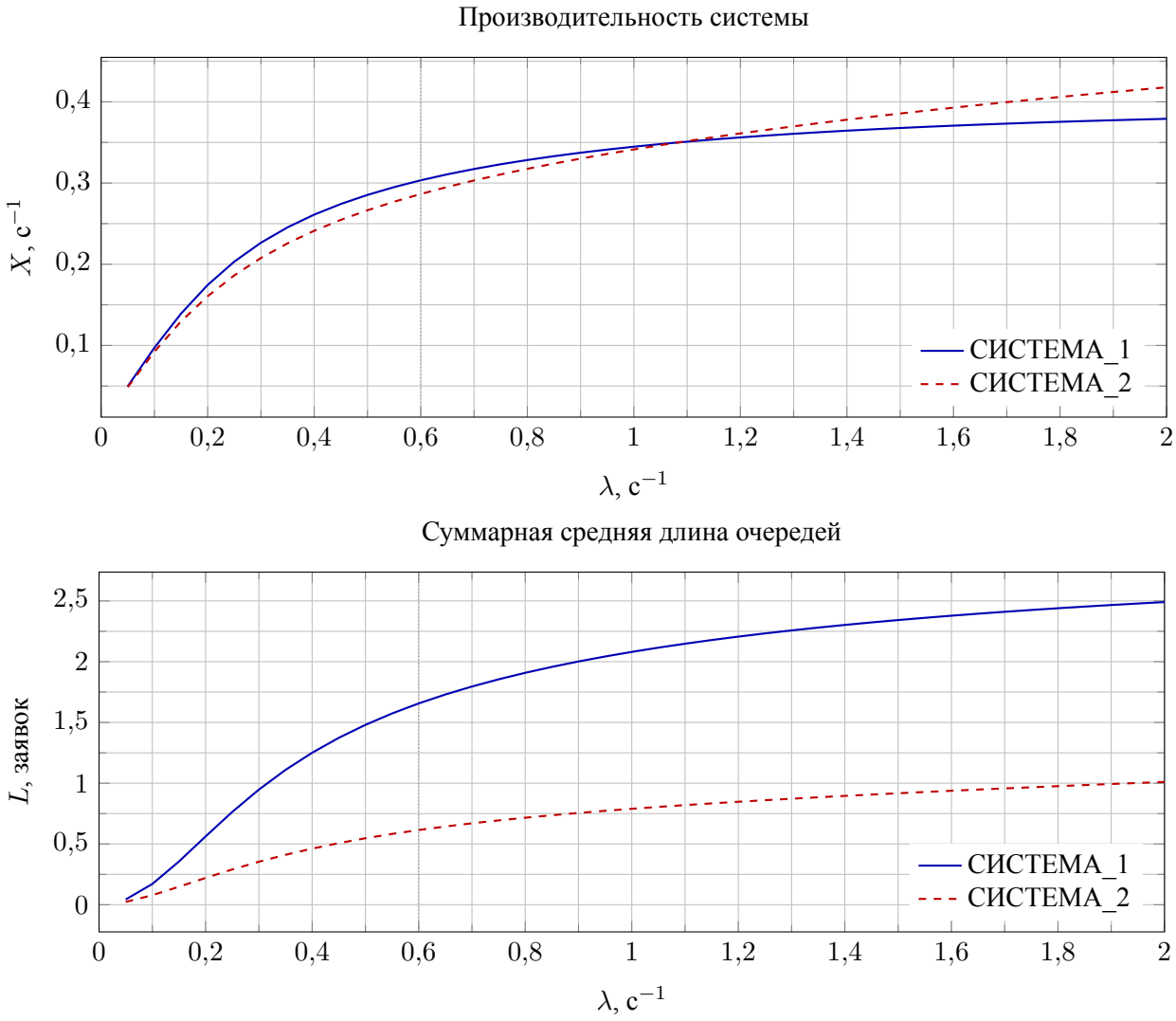


Рис. 7. Производительность и суммарная средняя длина очередей при варьировании λ.

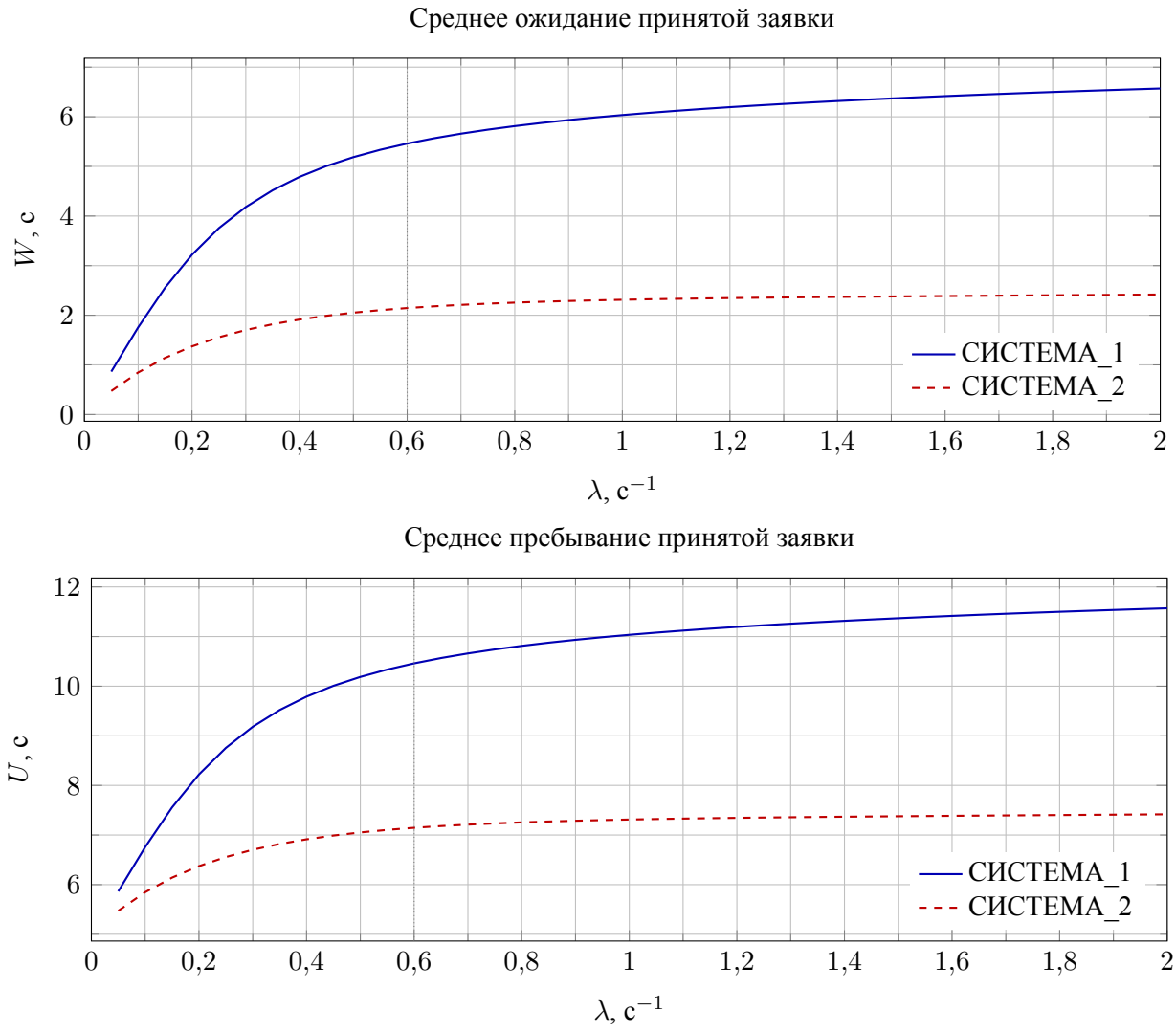


Рис. 8. Времена ожидания и пребывания принятых заявок, рассчитанные прямым усреднением по состояниям.

8.2. Объяснение различий и влияние накопителей

Обе системы направляют 75% входного потока к П1: его предлагаемая нагрузка $y_1 = 2{,}25$ существенно больше единицы. Во второй системе третий прибор получает лишь 5% потока; свободная мощность П2 и П3 не может обслужить заявки, потерянные у П1. Поэтому большее число приборов само по себе не обеспечивает большую суммарную загрузку при заданном правиле направления.

Преимущество первой системы при исходном λ связано с большим накопителем перед перегруженным П1 и наличием накопителя перед П2. У первой системы П2 принимает 25% потока и имеет одно место ожидания; у второй --- только 20% и работает без накопителя. Это более чем компенсирует наличие малозагруженного П3 во второй системе.

Влияние ёмкости нельзя оценивать путём прямого сравнения разных законов обслуживания и разного числа приборов. Для отдельной проверки рассмотрен *один и тот же* экспоненциальный П1 с $\lambda_1 = 0{,}45\,\text{с}^{-1}$, $b = 5$ с и разным числом мест ожидания:

Таблица 10. Контроль влияния накопителя при неизменных остальных параметрах

Места ожидания	Потери π_1	Загрузка ρ_1	Ожидание w_1, с
0	0,692308	0,692308	0,000000
1	0,609023	0,879699	3,461538
2	0,578113	0,949247	7,443609

Увеличение накопителя в данном контроле **уменьшает потери и повышает загрузку**, одновременно увеличивая среднее ожидание принятых заявок. Поэтому вывод о том, что накопитель только увеличивает ожидание и не снижает потери, для рассматриваемых моделей неверен.

Чтобы отделить эффект распределения обслуживания, сравним при *одном месте ожидания* экспоненциальный и эрланговский П1. Замена экспоненциального обслуживания на E_2 с тем же средним снижает потери с $0{,}609023$ до $0{,}595380$, повышает загрузку с $0{,}879699$ до $0{,}910396$ и уменьшает ожидание с $3{,}461538$ до $3{,}269896$ с. Этот выигрыш второй системы не устраняет преимущество первой по суммарной загрузке при исходной интенсивности.

8.3. Влияние выбора прибора с обслуживанием E_2

В соответствии с заданием закон Эрланга назначается ровно одному прибору, причём его можно выбрать. В основной модели выбран П1. Чтобы проверить зависимость вывода от этого решения, дополнительно рассчитаны все три допустимых размещения E_2 при исходной интенсивности. Этот контроль выполнен программой на Python (функция `placement_study`); основной расчёт WinMark и его графики относятся только к E_2 в П1.

Во всех трёх случаях сохранены накопители $1/0/1$, вероятности направления $(0{,}75; 0{,}20; 0{,}05)$ и среднее обслуживание $b = 5$ с. Меняется только прибор с двумя последовательными фазами. Если выбран П1 или П3, число состояний равно $5 \cdot 2 \cdot 3 = 30$ или $3 \cdot 2 \cdot 5 = 30$. Если выбран П2 без накопителя, его локальные состояния --- свободен, первая фаза, вторая фаза; полная модель содержит $3 \cdot 3 \cdot 3 = 27$ состояний.

Таблица 11. Контроль размещения E_2 во второй системе при $\lambda = 0{,}6\,\text{с}^{-1}$

Прибор с E_2	Состояний	Загрузка R	Производительность X	Потери π
П1	30	1,432517	0,286503	0,522494
П2	27	1,401821	0,280364	0,532726
П3	30	1,402434	0,280487	0,532522

Прибор П2 не имеет очереди. При сохранении среднего обслуживания его вероятность занятости равна $\lambda_2 b/(1+\lambda_2 b) = 0{,}375$ как для экспоненциального, так и для эрланговского закона; принятое поступление начинает обслуживание сразу. Поэтому перенос E_2 в П2 не даёт выигрыша от уменьшения разброса времени обслуживания в очереди.

Среди трёх размещений наибольшая суммарная загрузка второй системы достигается при E_2 в П1: $R = 1{,}432517$. Но и она меньше $R^{(1)} = 1{,}516814$. Следовательно, **при исходной нагрузке выбор первой системы сохраняется при любом допустимом выборе прибора с** E_2. Положение точки пересечения при варьировании, найденное выше, относится именно к основному размещению E_2 в П1.

9. Выбор наилучшего варианта организации системы

По заданному критерию максимальной загрузки при $\lambda = 0{,}6\,\text{с}^{-1}$ выбирается **СИСТЕМА_1**:

$$R^{(1)} = 1{,}516814 > R^{(2)} = 1{,}432517.$$

Абсолютный выигрыш составляет $0{,}084297$ одновременно работающего прибора в среднем, относительный --- $5{,}88\%$ относительно второй системы. Контроль размещения E_2 подтвердил этот выбор для всех трёх допустимых приборов второй системы (таблица 11). Средняя загрузка одного прибора также выше: $0{,}758407 > 0{,}477506$, поэтому выбор при исходных данных одинаков для суммарного и нормированного показателей.

Первая система обслуживает $0{,}303363$ заявки в секунду против $0{,}286503$; потери ниже на $2{,}809905$ процентного пункта ($49{,}44\%$ против $52{,}25\%$). При этом она имеет более длинные очереди и большее время пребывания: $10{,}460153$ с против $7{,}145069$ с. Следовательно, вторая система лучше по временным показателям, однако эти показатели не являются критерием данного варианта.

10. Выводы

Построены полные марковские модели варианта 33/73/11: двухприборная СИСТЕМА_1 с накопителями $2/1$ и трёхприборная СИСТЕМА_2 с накопителями $1/0/1$, в которой только П1 имеет обслуживание Эрланга второго порядка. Получены размеченные графы, генераторы размером 12×12 и 30×30, стационарные вероятности и все характеристики формы 2.

Расчёты выполнены в WinMark и подтверждены снимками окон, полными HTML-отчётами и экспортированными графиками. Независимая программа на Python проверила все 40 точек варьирования: 2240 значений характеристик и 1680 вероятностей; максимальное расхождение с WinMark меньше 10^{-5}. Времена ожидания и пребывания определены через распределение состояний в момент поступления принятой заявки и остаточное число фаз обслуживания. Формулы Литтла применены после расчёта для проверки. Стационарные вероятности дополнительно сверены с независимым аналитическим решением для отдельных приборов; максимальные расхождения находятся на уровне машинной точности.

При заданном $\lambda = 0{,}6\,\text{с}^{-1}$ лучшей по максимальной суммарной загрузке является СИСТЕМА_1 ($R = 1{,}516814$ против $1{,}432517$), хотя СИСТЕМА_2 обеспечивает меньшее время пребывания. В диапазоне варьирования победитель по суммарной загрузке меняется около $\lambda = 1{,}083188\,\text{с}^{-1}$. Дополнительная проверка всех трёх допустимых размещений E_2 сохранила выбор первой системы при исходной нагрузке. Контрольный расчёт при неизменных остальных параметрах подтвердил, что накопитель способен снижать потери и повышать загрузку ценой дополнительного ожидания.

Список использованных материалов

[1] Учебно-исследовательская работа 2 (УИР 2). «Марковские модели систем массового обслуживания». Университет ИТМО, факультет ПИ и КТ. Задание от 23.09.2024. 7 с.

[2] Основы моделирования дискретных систем. Раздел 5 «Численное моделирование (модели случайных процессов)». Предоставленный теоретический материал «МП_теория к УИР2». 68 с.

[3] Модели марковских случайных процессов (численное моделирование). Университет ИТМО, ФПИиКТ. Предоставленная презентация «ЧИСЛЕННОЕ МОДЕЛИРОВАНИЕ». 14 слайдов.

[4] WinMark. Результаты расчёта СИСТЕМА_1 (вариант 33), экспорт от 27.10.2026. Электронное приложение: `system1/report_system1.html`.

[5] WinMark. Результаты расчёта СИСТЕМА_2 (вариант 73), экспорт от 27.10.2026. Электронное приложение: `system2/report_system2.html`.

A. Полная численная матрица второй системы

Таблица задаёт все элементы G_2 при исходной нагрузке. В строке i отдельно указан g_{ii}, затем пары $j : g_{ij}$ для *всех* ненулевых внедиагональных элементов. Остальные элементы равны нулю. Это та же матрица, что в (2); порядок состояний соответствует таблице 2.

Таблица 12. Матрица G_2 в полном разреженном представлении

i	g_{ii}	Ненулевые внедиагональные элементы $j : g_{ij}$, c^{-1}
0	−0,60	1 : 0,03; 3 : 0,12; 6 : 0,45
1	−0,80	0 : 0,20; 2 : 0,03; 4 : 0,12; 7 : 0,45
2	−0,77	1 : 0,20; 5 : 0,12; 8 : 0,45
3	−0,68	0 : 0,20; 4 : 0,03; 9 : 0,45
4	−0,88	1 : 0,20; 3 : 0,20; 5 : 0,03; 10 : 0,45
5	−0,85	2 : 0,20; 4 : 0,20; 11 : 0,45
6	−1,00	7 : 0,03; 9 : 0,12; 12 : 0,40; 18 : 0,45
7	−1,20	6 : 0,20; 8 : 0,03; 10 : 0,12; 13 : 0,40; 19 : 0,45
8	−1,17	7 : 0,20; 11 : 0,12; 14 : 0,40; 20 : 0,45
9	−1,08	6 : 0,20; 10 : 0,03; 15 : 0,40; 21 : 0,45
10	−1,28	7 : 0,20; 9 : 0,20; 11 : 0,03; 16 : 0,40; 22 : 0,45
11	−1,25	8 : 0,20; 10 : 0,20; 17 : 0,40; 23 : 0,45
12	−1,00	0 : 0,40; 13 : 0,03; 15 : 0,12; 24 : 0,45
13	−1,20	1 : 0,40; 12 : 0,20; 14 : 0,03; 16 : 0,12; 25 : 0,45
14	−1,17	2 : 0,40; 13 : 0,20; 17 : 0,12; 26 : 0,45
15	−1,08	3 : 0,40; 12 : 0,20; 16 : 0,03; 27 : 0,45
16	−1,28	4 : 0,40; 13 : 0,20; 15 : 0,20; 17 : 0,03; 28 : 0,45
17	−1,25	5 : 0,40; 14 : 0,20; 16 : 0,20; 29 : 0,45
18	−0,55	19 : 0,03; 21 : 0,12; 24 : 0,40
19	−0,75	18 : 0,20; 20 : 0,03; 22 : 0,12; 25 : 0,40
20	−0,72	19 : 0,20; 23 : 0,12; 26 : 0,40
21	−0,63	18 : 0,20; 22 : 0,03; 27 : 0,40
22	−0,83	19 : 0,20; 21 : 0,20; 23 : 0,03; 28 : 0,40
23	−0,80	20 : 0,20; 22 : 0,20; 29 : 0,40
24	−0,55	6 : 0,40; 25 : 0,03; 27 : 0,12
25	−0,75	7 : 0,40; 24 : 0,20; 26 : 0,03; 28 : 0,12
26	−0,72	8 : 0,40; 25 : 0,20; 29 : 0,12
27	−0,63	9 : 0,40; 24 : 0,20; 28 : 0,03
28	−0,83	10 : 0,40; 25 : 0,20; 27 : 0,20; 29 : 0,03
29	−0,80	11 : 0,40; 26 : 0,20; 28 : 0,20

B. Программа расчёта всех характеристик

Ниже приведён полный код независимого расчёта, использованный для проверки WinMark, построения таблиц и контрольных экспериментов. Требуется Python 3.10 или новее; сторонние библиотеки не нужны. Код хранится в `../app/calculate_uir2.py` и подключается к отчёту без создания копий. Для повторения вычислений из корня папки лабораторной работы нужно выполнить:

```
python app/calculate_uir2.py
```

Программа выводит все вероятности, характеристики и опорные данные в консоль. Файлы `../out/results.json` и `../out/calculation_log.txt` автоматически сохраняются в папке `out` проекта независимо от рабочей папки запуска. Запуск Python требуется только для повторной проверки чисел. Для компиляции отчёта из папки `docs` нужно сохранить структуру проекта: снимки окон находятся в `evidence`, исходные графики --- в `system1/graphics_system1` и `system2/graphics_system2`, код --- в `../app`. Сравнительные кривые и численные таблицы встроены в `.tex`. Функции `metrics` и `calculate` показывают соответственно прямой расчёт показателей и последующую независимую проверку.

Listing 1. Полный исходный код расчёта варианта 33/73/11

```
"""Variant 33/73/11. Python 3.10+, standard library only.

Run from project root: python app/calculate_uir2.py
Outputs are always written to the project out/ directory.
State: tuple of local (number_in_node, service_phase) pairs.
K includes the job in service; buffer capacity is K - 1.
"""
from itertools import product
from math import prod
import json
import io
from contextlib import redirect_stdout
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
OUTPUT_DIR = PROJECT_ROOT / "out"

B = 5.0
MU = 1.0 / B
CONFIGS = [((3, 2), (1, 1), (0.75, 0.25)),
           ((2, 1, 2), (2, 1, 1), (0.75, 0.20, 0.05))]


def solve(a, rhs):
    """Gaussian elimination with partial pivoting."""
    n = len(rhs)
    aug = [list(row) + [rhs[i]] for i, row in enumerate(a)]
    for j in range(n):
        k = max(range(j, n), key=lambda i: abs(aug[i][j]))
        assert abs(aug[k][j]) > 1e-14
        aug[j], aug[k] = aug[k], aug[j]
        for i in range(j + 1, n):
            r = aug[i][j] / aug[j][j]
            for k in range(j + 1, n + 1):
                aug[i][k] -= r * aug[j][k]
            aug[i][j] = 0.0
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (aug[i][-1] - sum(aug[i][j] * x[j]
                                 for j in range(i + 1, n))) / aug[i][i]
    return x


def local_states(capacity, phases):
    return [(0, 0)] + [(n, f) for n in range(1, capacity + 1)
                       for f in range(1, phases + 1)]
```

```
def generator(capacities, phases, rates):
    spaces = [local_states(k, f) for k, f in zip(capacities, phases)]
    states = list(product(*spaces))
    index = {s: i for i, s in enumerate(states)}
    q = [[0.0] * len(states) for _ in states]
    for i, state in enumerate(states):
        for j, (n, f) in enumerate(state):
            moves = []
            if n < capacities[j]:
                target = (1, 1) if n == 0 else (n + 1, f)
                moves.append((target, rates[j]))
            if n > 0:
                if f < phases[j]:
                    target = (n, f + 1)
                else:
                    target = (0, 0) if n == 1 else (n - 1, 1)
                moves.append((target, phases[j] / B))
            for target, rate in moves:
                changed = list(state)
                changed[j] = target
                q[i][index[tuple(changed)]] += rate
        q[i][i] = -sum(q[i])
    return states, q


def stationary(q):
    n = len(q)
    a = [[q[j][i] for j in range(n)] for i in range(n)]
    a[-1] = [1.0] * n
    return solve(a, [0.0] * (n - 1) + [1.0])


def metrics(states, p, capacities, phases, rates):
    nodes = []
    for j, (cap, k, lam) in enumerate(zip(capacities, phases, rates)):
        busy = length = number = loss = output = accept = 0.0
        wait_mass = stay_mass = 0.0
        for state, prob in zip(states, p):
            n, f = state[j]
            busy += prob * (n > 0)
            length += prob * max(0, n - 1)
            number += prob * n
            loss += prob * (n == cap)
            output += prob * (k / B if n > 0 and f == k else 0.0)
            if n < cap:
                # Conditional waiting time given the state seen on arrival.
                # Current service: remaining phases; waiting jobs: full B.
                tw = 0.0 if n == 0 else (k - f + 1) * B / k + (n - 1) * B
                tu = tw + B
                accept += prob
                wait_mass += prob * tw
                stay_mass += prob * tu
        nodes.append(dict(y=lam * B, rho=busy, l=length, m=number,
                          pi=loss, x=output, w=wait_mass / accept,
                          u=stay_mass / accept, a=accept,
                          wait_mass=wait_mass, stay_mass=stay_mass))
    accepted = sum(lam * d['a'] for lam, d in zip(rates, nodes))
    total = {key: sum(d[key] for d in nodes)
             for key in ('y', 'rho', 'l', 'm', 'x')}
    total['pi'] = sum(lam * d['pi'] for lam, d in zip(rates, nodes)) / sum(rates)
    total['w'] = sum(lam * d['wait_mass'] for lam, d in zip(rates, nodes)) / accepted
    total['u'] = sum(lam * d['stay_mass'] for lam, d in zip(rates, nodes)) / accepted
    total['rho_mean'] = total['rho'] / len(nodes)
    return nodes, total


def analytic_local(cap, phases, lam):
    if phases == 1:
        weights = [(lam * B) ** n for n in range(cap + 1)]
    else:
        assert (cap, phases) == (2, 2)
        t = lam * B / 2.0
        weights = [1.0, t * (1 + t), t,
                   t * t * (1 + t), t * t * (2 + t)]
    return [w / sum(weights) for w in weights]
```

```
def calculate(system, intensity):
    caps, phases, weights = CONFIGS[system]
    rates = [intensity * w for w in weights]
    states, q = generator(caps, phases, rates)
    p = stationary(q)
    nodes, total = metrics(states, p, caps, phases, rates)
    row_err = max(abs(sum(row)) for row in q)
    residual = max(abs(sum(p[i] * q[i][j] for i in range(len(p))))
                   for j in range(len(p)))
    local = [analytic_local(c, f, a) for c, f, a in zip(caps, phases, rates)]
    spaces = [local_states(c, f) for c, f in zip(caps, phases)]
    factor_err = max(abs(prob - prod(local[j][spaces[j].index(s)]
                                     for j, s in enumerate(state)))
                     for state, prob in zip(states, p))
    checks = [abs(sum(p) - 1), row_err, residual, factor_err]
    for lam, d in zip(rates, nodes):
        checks += [abs(d['x'] - lam * (1 - d['pi'])),
                   abs(d['rho'] - B * d['x']),
                   abs(d['l'] - d['x'] * d['w']),
                   abs(d['m'] - d['x'] * d['u']),
                   abs(d['u'] - d['w'] - B)]
    checks += [abs(total['l'] - total['x'] * total['w']),
               abs(total['m'] - total['x'] * total['u'])]
    assert min(p) >= -1e-12 and max(checks) < 1e-10
    return dict(states=states, q=q, p=p, nodes=nodes, total=total,
                local=local, errors=dict(rows=row_err, stationary=residual,
                                         factorization=factor_err, all=max(checks)))


def placement_study(intensity=0.6):
    """Move E2 service to exactly one of the three servers."""
    caps, _, weights = CONFIGS[1]
    rates = [intensity * w for w in weights]
    results = []
    for chosen in range(3):
        phases = [1, 1, 1]
        phases[chosen] = 2
        states, q = generator(caps, phases, rates)
        p = stationary(q)
        nodes, total = metrics(states, p, caps, phases, rates)
        checks = [abs(sum(p) - 1)]
        checks += [abs(sum(p[i] * q[i][j] for i in range(len(p))))
                   for j in range(len(p))]
        for lam, d in zip(rates, nodes):
            checks += [abs(d['x'] - lam * d['a']),
                       abs(d['l'] - d['x'] * d['w']),
                       abs(d['m'] - d['x'] * d['u'])]
        assert min(p) > 0 and max(checks) < 1e-10
        results.append(dict(server=chosen + 1, states=len(states),
                            total=total, error=max(checks)))
    return results


def crossing():
    left, right = 1.0, 2.0
    for _ in range(50):
        mid = (left + right) / 2
        delta = calculate(0, mid)['total']['rho'] - calculate(1, mid)['total']['rho']
        if delta > 0:
            left = mid
        else:
            right = mid
    return (left + right) / 2


def print_result(label, result):
    print('\n' + label, 'states =', len(result['states']))
    print('i   state                          probability')
    for i, (s, p) in enumerate(zip(result['states'], result['p'])):
        print(f'{i:2d}  {str(s):28s}  {p:.12f}')
    print('node      y        rho        l          m         pi         X          W          U')
    for name, d in list(enumerate(result['nodes'], 1)) + [('ALL', result['total'])]:
        print(f'{str(name):>4s} ' + ' '.join(f'{d[k]:10.6f}'
              for k in ('y', 'rho', 'l', 'm', 'pi', 'x', 'w', 'u')))
    print('max errors:', ' '.join(f'{k}={v:.3e}' for k, v in result['errors'].items()))
```

```python


def main():
    print('Python', sys.version.split()[0])
    print('Variant 33/73/11; lambda=0.6; B=5; mu=0.2; nu=0.4')
    baseline = [calculate(i, 0.6) for i in range(2)]
    for i, result in enumerate(baseline, 1):
        print_result(f'SYSTEM_{i}', result)
    grid = [n / 20 for n in range(1, 41)]
    sweep = [(a, calculate(0, a)['total'], calculate(1, a)['total']) for a in grid]
    print('\nlambda       R1         R2        pi1        pi2         U1         U2')
    for a, x, y in sweep:
        print(f'{a:.2f} ' + ' '.join(f'{v:.8f}' for v in
              (x['rho'], y['rho'], x['pi'], y['pi'], x['u'], y['u'])))
    cross = crossing()
    print(f'R1=R2 at lambda={cross:.9f}')
    # Additional controls isolate buffer and service-distribution effects.
    controls = []
    for cap, phases in [(2, 2), (2, 1), (3, 1), (1, 1)]:
        states, q = generator((cap,), (phases,), (0.45,))
        ps = stationary(q)
        _, t = metrics(states, ps, (cap,), (phases,), (0.45,))
        controls.append(dict(K=cap, phases=phases, total=t))
    print('\nCONTROL: first node, arrival=0.45')
    for d in controls:
        print(f"K={d['K']}, phases={d['phases']}: " +
              ' '.join(f'{k}={d["total"][k]:.9f}' for k in ('rho', 'pi', 'x', 'w', 'u')))
    placement = placement_study()
    print('\nE2 PLACEMENT: system 2, lambda=0.6')
    for d in placement:
        print(f"server={d['server']}, states={d['states']}: " +
              ' '.join(f'{k}={d["total"][k]:.9f}'
                       for k in ('rho', 'pi', 'x', 'w', 'u')))
    (OUTPUT_DIR / 'results.json').write_text(json.dumps(dict(baseline=baseline,
        sweep=sweep, crossing=cross, controls=controls, placement=placement),
        indent=2), encoding='utf8')


if __name__ == '__main__':
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    output = io.StringIO()
    try:
        with redirect_stdout(output):
            main()
    finally:
        log = output.getvalue()
        (OUTPUT_DIR / 'calculation_log.txt').write_text(log, encoding='utf8')
        print(log, end='')
```

C. Протокол фактического выполнения

Ниже приведён фрагмент реального консольного вывода той же программы: заголовки запусков, все восемь характеристик каждого прибора и системы, максимальные невязки и контрольные расчёты. Полные вероятности и результаты варьирования из вывода перенесены в таблицы основной части отчёта; они также сохраняются программой в `results.json`.

```
Python 3.12.14
Variant 33/73/11; lambda=0.6; B=5; mu=0.2; nu=0.4

SYSTEM_1 states = 12
node      y        rho        l          m         pi         X          W          U
   1   2.250000   0.949247   1.413164   2.362411   0.578113   0.189849   7.443609  12.443609
   2   0.750000   0.567568   0.243243   0.810811   0.243243   0.113514   2.142857   7.142857
 ALL   3.000000   1.516814   1.656407   3.173222   0.494395   0.303363   5.460153  10.460153
max errors: rows=5.551e-17 stationary=4.510e-17 factorization=8.674e-17 all=1.776e-15

SYSTEM_2 states = 30
node      y        rho        l          m         pi         X          W          U
   1   2.250000   0.910396   0.595380   1.505775   0.595380   0.182079   3.269896   8.269896
   2   0.600000   0.375000   0.000000   0.375000   0.375000   0.075000   0.000000   5.000000
   3   0.150000   0.147122   0.019190   0.166311   0.019190   0.029424   0.652174   5.652174
 ALL   3.000000   1.432517   0.614570   2.047087   0.522494   0.286503   2.145069   7.145069
max errors: rows=8.327e-17 stationary=3.014e-17 factorization=3.469e-17 all=1.776e-15

R1=R2 at lambda=1.083188077

CONTROL: first node, arrival=0.45
K=2, phases=2: rho=0.910395520 pi=0.595379769 x=0.182079104 w=3.269896194 u=8.269896194
K=2, phases=1: rho=0.879699248 pi=0.609022556 x=0.175939850 w=3.461538462 u=8.461538462
K=3, phases=1: rho=0.949246630 pi=0.578112609 x=0.189849326 w=7.443609023 u=12.443609023
K=1, phases=1: rho=0.692307692 pi=0.692307692 x=0.138461538 w=0.000000000 u=5.000000000

E2 PLACEMENT: system 2, lambda=0.6
server=1, states=30: rho=1.432517055 pi=0.522494315 x=0.286503411 w=2.145068822 u=7.145068822
server=2, states=27: rho=1.401820783 pi=0.532726406 x=0.280364157 w=2.240701270 u=7.240701270
server=3, states=30: rho=1.402434087 pi=0.532521971 x=0.280486817 w=2.225144253 u=7.225144253
```

В протоколе `rho` означает суммарную загрузку для строки `ALL`, `X` --- производительность, `W`/`U` --- прямые временные характеристики. Все значения напечатаны после успешного прохождения проверок `assert`; при нарушении допуска 10^{-10} программа прерывается. Погрешности в строках `max errors` показаны до округления табличных результатов.

D. Протокол сверки HTML-отчётов WinMark

Протокол получен запуском `verify_winmark.py`. Скрипт читает оба реальных HTML-экспорта, проверяет матрицы и параметры, сопоставляет результаты в исходной точке и во всех 40 точках с независимым решением, а также проверяет точные выражения формул на независимо рассчитанных вероятностях. Скрипт находится в папке `docs`; файлы `winmark_verification.json` и `winmark_verification.txt` сохраняются рядом с ним автоматически. Команда повторной проверки из корня папки лабораторной работы:

```
python docs/verify_winmark.py
```

```
SYSTEM_1: states=12; formulas=24; steps=40
checked sweep: 960 metric values; 480 probabilities
matrix: max absolute difference=1.110223025e-16
point_metrics: max absolute difference=2.977443609e-06
point_probabilities: max absolute difference=4.527509270e-07
sweep_metrics: max absolute difference=7.913210814e-06
sweep_probabilities: max absolute difference=5.202213989e-07
formula_values: max absolute difference=2.131628207e-14
probability_normalization: max absolute difference=2.000000000e-06
point: Y=3.000000 R=1.516814 Q=1.656407 M=3.173222 W=5.460153 U=10.460154 Loss=0.494395 Perf
    =0.303363
SYSTEM_2: states=30; formulas=32; steps=40
checked sweep: 1280 metric values; 1200 probabilities
matrix: max absolute difference=1.110223025e-16
point_metrics: max absolute difference=9.130434790e-07
point_probabilities: max absolute difference=4.949851970e-07
sweep_metrics: max absolute difference=7.055259429e-06
sweep_probabilities: max absolute difference=5.286955110e-07
formula_values: max absolute difference=7.105427358e-15
probability_normalization: max absolute difference=4.000000000e-06
point: Y=3.000000 R=1.432517 Q=0.614569 M=2.047086 W=2.145069 U=7.145069 Loss=0.522494 Perf=0.286503
PASS: both exports agree with independent calculations.
```